TP01
Análisis de Texto

Matias Rolón
143354

1. *Utilizando la colección para debugging[1] provista por el equipo docente escriba un programa que realice el análisis léxico sobre la misma y extraiga la siguiente información:*

- *Lista de términos y su DF*
- *Cantidad de tokens*
- *Cantidad de términos*
- *Cantidad de documentos procesados*

Compare los resultados obtenidos por su programa con los que figuran en el archivo collection_data.json que contiene los metadatos de la colección.

Respuesta
Para este punto, el factor más determinante que hacía variar el número de tokens encontrados fue la expresión regular usada en el *tokenizer*. Para la misma se hizo una expresión que detecte todo lo que no sea un carácter alfanumérico, y se lo elimina del string.
Expresión regular: '[^\w\s]|_'
Aclaración: ***\w*** por defecto incluye letras, números y el guión bajo. Por eso al final de la expresión declaramos explícitamente el guión bajo, para que luego sea eliminado.

Se comparan los datos obtenidos por el programa propio con respecto a los que figuran en el archivo *collection_data.json*:

	Programa	collection_data
Cantidad de tokens	793.254	793.254
Cantidad de términos	20	20
Cantidad de documentos procesados	10.000	10.000

Tabla 1: estadísticas.

Todos los datos coinciden.

[1] https://drive.google.com/file/d/1-Oz6ffJoKsTZM5S4gaPx6nzrxfVOVOG_/view?usp=sharing

2. *Escriba un programa que realice análisis léxico sobre la colección RI-tknz-data[2]. El programa debe recibir como parámetros el directorio donde se encuentran los documentos y un argumento que indica si se deben eliminar las palabras vacías (y en tal caso, el nombre del archivo que las contiene). Defina, además, una longitud mínima y máxima para los términos. Como salida, el programa debe generar:*

- *Un archivo (`terminos.txt`) con la lista de términos a indexar (ordenado), su frecuencia en la colección y su DF (Document Frequency).*
  Formato de salida: `$<termino> [ESP] <CF> [ESP] <DF>$`.
  Ejemplo:
  ```
  casa 238 3
  perro 644 6
  ...
  zorro 12 1
  ```
- *Un segundo archivo (`estadisticas.txt`) con los siguientes datos (un ítem por línea y separados por espacio cuando sean más de un valor):*
  - *Cantidad de documentos procesados.*
  - *Cantidad de tokens y términos extraídos.*
  - *Promedio de tokens y términos de los documentos.*
  - *Largo promedio de un término.*
  - *Cantidad de tokens y términos del documento más corto y del más largo[3].*
  - *Cantidad de términos que aparecen sólo 1 vez en la colección.*

- *Un tercer archivo (`frecuencias.txt`), con:*
  - *La lista de los 10 términos más frecuentes y su CF (Collection Frequency). Un término por línea.*
  - *La lista de los 10 términos menos frecuentes y su CF. Un término por línea.*

Respuesta

Se decidió dejar el mismo *tokenizer* del punto 1. A efectos de hacer hincapié en el análisis de las *stopwords* y no en procesos adicionales del *tokenizer*, que se agregarán en los puntos siguientes a fin de perfeccionarlo.

Seteamos un largo mínimo para los términos de 2 caracteres. Esto nos permitirá incorporar palabras como abreviaturas (c.p., c.v.), aunque sólo las que estén normalizadas (sin puntos), códigos de países como “ar”, entre otras.

---

[2] https://github.com/tolosoft-academia/RI_2025/tree/main/data

[3] Mida la longitud del documento en cantidad de *tokens*.

Por el lado del largo máximo, las palabras más largas en español de la Real Academia Española suelen medir entre 20 y 23 letras, por lo que un máximo de 25 caracteres podrá ocupar la mayoría de las palabras largas que se mencionan en todos los documentos.

Podemos destacar, dentro de los 10 términos más frecuentes, que todas son propias del dominio en el que está enmarcada la colección, es decir, el académico. Así, destacan palabras como “universidad”, “educación”, “facultad”, “ciencias”, “carreras”, etc.
Como también dominios de nivel superior pertenecientes a URLs (“edu”, “ar”).
Si vemos el archivo de estadísticas generado, podemos ver que se procesaron 498 archivos, y se extrajeron 26.677 términos, de los cuales aproximadamente la mitad (13019) sólo aparece una vez.

3. *Escriba un segundo tokenizer que implemente los criterios del artículo de Grefenstette y Tapanainen para definir qué es una “palabra” (o término) y cómo tratar números y signos de puntuación. En este caso su tokenizer deberá extraer y tratar como un único término:*

- *Abreviaturas tal cual están escritas (por ejemplo, Dr., Lic., S.A., etc.)*[4]
- *Direcciones de correo electrónico y URLs.*
- *Números (por ejemplo, cantidades, teléfonos).*
- *Nombres propios (por ejemplo, Villa Carlos Paz, Manuel Belgrano, etc.)*

Utilice la colección para debugging[5] *de expresiones regulares provista por el equipo docente para extraer y comparar la salida de su programa con los metadatos de la colección tal como lo realizó en el punto 1.*

Por último, extraiga y almacene la misma información que en el punto 2 sobre la colección RI-tknz-data utilizando su nuevo tokenizer.

Respuesta
Tomando como base el paper de Grefenstette[6], implementaremos varios filtros diferentes para los distintos tipos de datos, de manera escalonada, con una prioridad definida para cada uno.
Partiendo del contexto, al ser un corpus de documentos académicos se decide dejar como tokens válidos tanto los números, fechas y abreviaturas que puedan encontrarse, ya que una consulta a un sistema como este suele estar acompañada de necesidades de información que usen estos tipos de datos. Ejemplo: inscripción

[4] Discuta cómo resolvería la extracción de abreviaturas como "NASA".
[5] https://drive.google.com/file/d/1mP4t7HlDF66joGV9CPok8ZZxC1N-J5LT/view
[6] What is a word, What is a sentence? Problems of Tokenization

a fechas de exámenes, nombres de profesores (Lic., Ing., Prof.), mails de alumnos, etc.
A partir de esta suposición se decide primero separar estos tipos de tokens, enmascararlos en el texto (reemplazarlos por un string vacío), y luego en un último paso, encontrar las palabras comunes. El orden queda de la siguiente manera:

1. URLs
2. Mails
3. Fechas
4. Números en diferentes formatos.
5. Abreviaturas
6. Nombres propios. Se agregó una validación para que puedan existir algunos conectores en el medio entre las palabras con mayúsculas de los nombres propios. Ejemplo: Centro **de** Estudiantes.
7. Palabras comunes restantes.

	Programa	collection_data
Cant. tokens	74.983	74.983
Cant. términos	20	20
Cant. documentos procesados	1.000	1.000

Tabla 3: estadísticas Programa vs estadísticas Debugging.

De acuerdo a la comparación que realiza el programa, y las estadísticas finales del archivo *collection_data.json* de la colección de debugging, podemos ver los mismos resultados. El apartado donde se tuvo que definir un trade-off fue en las abreviaturas, ya que, si bien en el caso de debugging no se apreció, en próximas colecciones cualquier palabra corta (menos de 5 palabras) que termine en un punto será tomada como abreviatura. Ejemplo: “Ellos comían **pan.**”
Una alternativa para lidiar con esto es restringir esas abreviaciones a una lista fija de posibilidades, pero se acotará al contexto de la colección y dejará de ser una expresión regular genérica. Por este motivo, se decide aceptar **falsos positivos** en esta expresión, como el ejemplo mencionado.

Por último, al generar los mismos archivos que en el punto 2 sobre la colección RI-tknz-data con este nuevo *tokenizer* se observan las siguientes diferencias:

- Se detectan más de 8000 tokens nuevos.
- Se detectan más de 4000 términos nuevos.
- En el archivo de *terminos.txt* se ve claramente como la mayoría de esos 4000 términos nuevos tienen que ver con distintos formatos de números que ahora son captados antes de procesar el resto (montos, fechas, legajos).

- En el archivo de *frecuencias.txt* salieron del top 10 de términos más frecuentes aquellos que podrían formar parte de una URL ("edu", "ar") y fueron reemplazados por palabras comunes como "información". Además, al establecer años como términos, "2012" entra al top como uno de los más mencionados.

En definitiva, los distintos niveles de *regex* ejecutados, nos permitieron segmentar mejor los conceptos de cada documento. Palabras que con un primer *tokenizer* no significaban nada semánticamente, en esta nueva versión son tomados en su totalidad. "edu" no significa nada por sí sólo, más que un dominio de una url, pero en "mail@unlu.edu.ar" pasa a ser algo concreto (el mail de una oficina, una persona, etc.)

4. *A partir del programa del ejercicio 1, incluya un proceso de stemming[7]. Luego de modificar su programa, corra nuevamente el proceso del ejercicio 2 y analice los cambios en la colección. ¿Qué implica este resultado? Busque ejemplos de pares de términos que tienen la misma raíz pero que el stemmer los trató diferente y términos que son diferentes y se los trató igual.*

Respuesta:
Se corrió el proceso del ejercicio 2 (con colección *RI-tkzn-data*) utilizando el *tokenizer* básico del ejercicio 1 pero añadiendo un proceso de stemming con la librería SnowballStemmer en español.

	P2 (sin stemming)	P4 (Stemming)
Cant. documentos procesados	498	498
Cant. tokens	369.987	369.987
Cantidad de términos	26.953	17.231
Largo promedio de términos	8,68	7,39
Términos que aparecen sólo 1 vez en la colección	13.034	7.758

Tabla 4: estadísticas de programas puntos 2 y 4.

Por la naturaleza del stemming, distintas variantes morfológicas de una palabra se guardan en el vocabulario como una sola raíz. Eso explica la cantidad menor de

[7] Puede usar la librería NLTK (Natural Language Toolkit). Revise qué algoritmos soporta (español e inglés).

términos que hay en el resultado del Punto 4, 9.723 menos, lo que equivale a más de un 35% aplicando esta técnica.
A su vez, como consecuencia de ésto, si observamos el archivo *términos.txt* del Punto 4 vemos que muchos términos tienen mayor frecuencia, al sumar las frecuencias de cada palabra distinta que corresponda a una misma raíz.

Sin embargo no es un método infalible ya que tenemos algunos ejemplos donde se tiene la misma raíz y el stemmer los trata como diferentes términos:
- La palabra "*acceder*" se redujo a "*acced*"
- La palabra "*acceso*" se redujo a "*acces*"
Pero ambas tienen la misma raíz (*"acce"*).
Otro caso de error, se da con las palabras "*actividad*" y "*activo*" donde ambos son reducidos a "*activ*", sin embargo no necesariamente hacen referencia a lo mismo.

Por ende, podemos decir que el stemming reduce el tamaño del vocabulario ampliamente, y reduce términos similares a su raíz, lo que puede mejorar el recall de las queries en una segunda etapa, al haber más coincidencias (siempre y cuando tambien se pase por un stemmer a la query), pero **como contrapartida** a veces comete errores de precisión semántica y no unifica lo que debería, o unifica mal, desventaja que debe ser tenido en cuenta dependiendo el caso de uso.

5. *Sobre la colección Vaswani[8], ejecute los stemmers Porter y Lancaster provistos en el módulo* `nltk.stem`*. Compare: cantidad de tokens únicos resultantes, resultado 1 a 1 y tiempo de ejecución para toda la colección. ¿Qué conclusiones puede obtener de la ejecución de uno y otro?*

Respuesta:
Al realizar la ejecución de cada uno por separado obtenemos las siguientes métricas:

	Porter	Lancaster
Tiempo de ejecución	4,99 seg.	3,62 seg.
Tokens únicos	7993	6877
Stems iguales	7128 (58%)	
Stems diferentes	5061 (42%)	

Tabla 5: estadísticas por Stemmers.

[8] https://github.com/tolosoft-academia/RI_2025/tree/main/data El archivo de documentos es `./corpus/doc-text.trec` y está en un formato con tags para el número de doc (`<docno>`) y el texto a continuación.

Como conclusiones, podemos afirmar que Lancaster genera menos tokens únicos pero tarda menos en recorrer toda la colección (más de 1 segundo de diferencia para una colección chica). Mientras que Porter genera más tokens a un tiempo más lento de ejecución.
Luego, por lo que se analiza en el archivo de comparación 1 a 1, podemos ver que Lancaster agrupa más palabras en un mismo término raíz. Al ser "agresivo", corre el riesgo de **perder información** en mayor medida. Un claro ejemplo es el caso de "activity", donde Porter lo toma como "activ" pero Lancaster como "act" acortando demasiado la palabra. También otros casos como "accept", donde Porter lo toma tal cual pero Lancaster lo cambia a "acceiv", lo que resulta en la peor de las desventajas: modificar el significado completo del token.
En definitiva, Porter resulta más conservador al no tener tantos ejemplos de éstos y mantener la forma original del texto, pero realizando el trabajo de manera más lenta.

Finalmente, a nivel general tuvieron muchas diferencias en cómo realizaron los stems con más del 40% del total de tokens únicos procesados distinto, lo que quiere decir que no son métodos equivalentes. Se debe evaluar detenidamente las características de cada uno a la hora de definir cuál usar, ya que entregarán diferentes vocabularios finales.

6. *Escriba un programa que realice la identificación del lenguaje de un texto a partir de un conjunto de entrenamiento[9]. Pruebe dos métodos sencillos:*

- *Uno basado en la distribución de la frecuencia de las letras.*
- *El segundo, basado en calcular la probabilidad de que una letra x preceda a otra letra y (calcule una matriz de probabilidades con todas las combinaciones).*

Compare los resultados contra el módulo Python `langdetect`[10] *y la solución provista.*

Respuesta:

	Porcentaje de acierto (en test)
Frecuencia de Letras	93,67%
Matriz de Combinación	98,00%
Librería externa (*langdetect*)	99,00%

Tabla 6.1: % aciertos según método de detección de idioma.

[9] Utilice los datos de entrenamiento y de evaluación provistos en: https://github.com/tolosoft-academia/RI_2025/blob/main/data/languageIdentificationData.tar.gz
[10] https://pypi.org/project/langdetect/

Ambos métodos creados por código tienen menor porcentaje de aciertos que la librería externa *langdetect,* lógico tratándose de un modelo más complejo. Pero cabe destacar que el método basado en las probabilidades de cada combinación logra resultados muy similares a la librería. Se diferencia del método de frecuencia de letras porque a comparación de éste, puede detectar patrones (que un carácter preceda a otro), factor importante ya que cada idioma tiene los suyos.
Sin embargo, el porcentaje de aciertos del método basado en la distribución de letras no tiene malos resultados, ya que sigue siendo arriba de un 90%. Para ser la solución más simple de las tres, sigue siendo confiable.

Idioma / Modelo	Frecuencia de Letras	Matriz de Combinación	Librería *langdetect*
Inglés	7	4	0
Francés	2	2	0
Italiano	10	0	0

Tabla 6.2: Errores por idioma según método.

7. *En este ejercicio se propone verificar la predicción de ley de Zipf. Para ello, descargue desde Project Gutenberg el texto del Quijote de Cervantes*[11] *y escriba un programa que extraiga los términos y calcule sus frecuencias (el programa debe generar la lista ordenada por frecuencia descendente). Calcule la curva de ajuste utilizando la función Polyfit del módulo NumPy*[12]*. Con los datos crudos y los estimados grafique en la notebook ambas distribuciones (haga 2 gráficos, uno en escala lineal y otro en log-log). ¿Cómo se comporta la predicción? ¿Qué conclusiones puede obtener?*

Respuesta:
En el **gráfico lineal** se observa un descenso repentino de la serie al inicio, es decir, pocas palabras con frecuencia muy alta y muchas palabras con frecuencia baja. El ajuste obtenido captura la tendencia general.
Se deduce que la ley de Zipf se cumple para el conjunto de datos indicado.

[11] http://www.gutenberg.org/cache/epub/2000/pg2000.txt (en UTF-8)
[12] https://docs.scipy.org/doc/numpy-1.15.0/reference/generated/numpy.polyfit.html

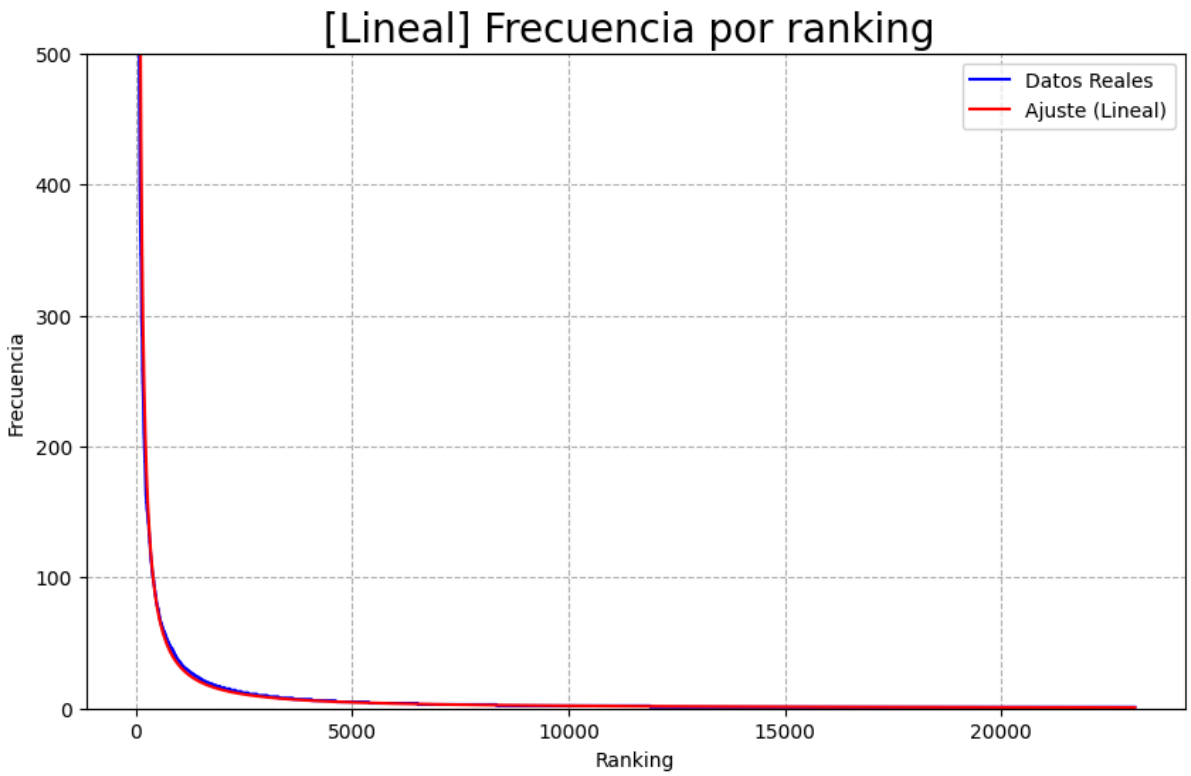

Gráfico 7.1: Frecuencia por ranking. Lineal

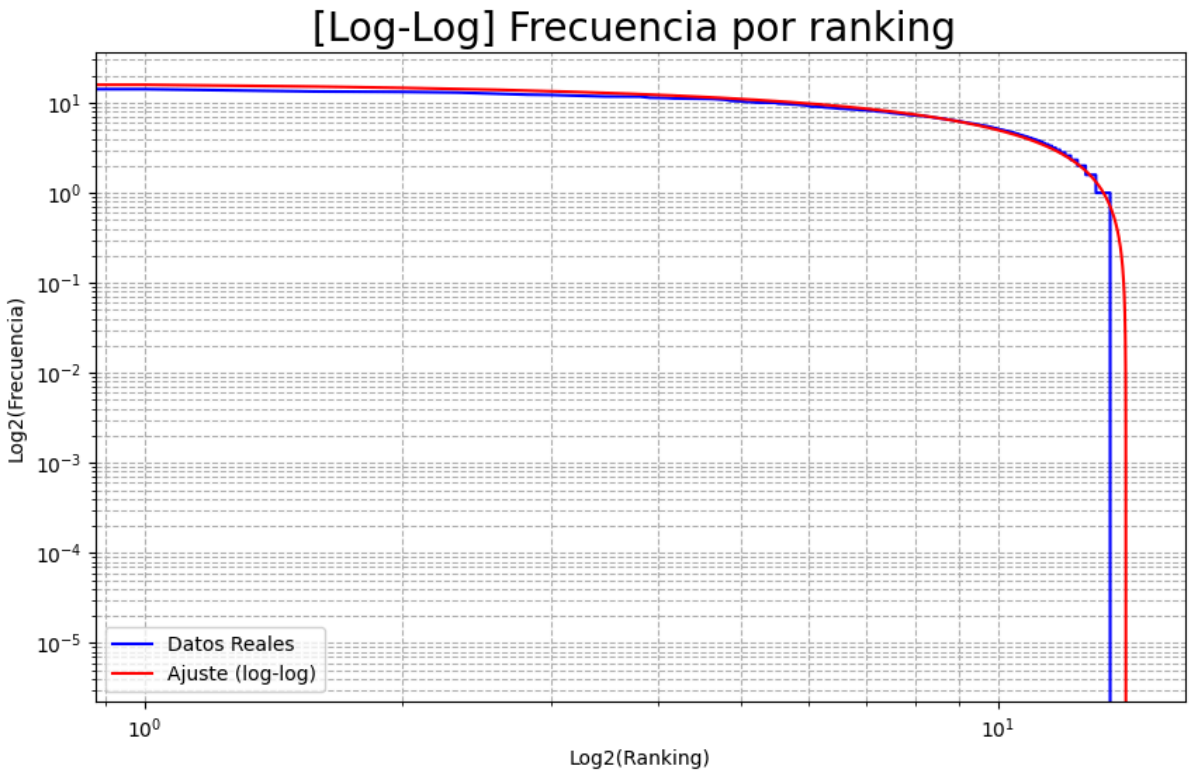

Gráfico 7.2: Frecuencia por ranking. Escala logaritmica

Por el lado del gráfico log-log podemos observar que es una serie decreciente. A su vez se aprecia similitud entre las dos series en el medio del gráfico, mientras que en

los extremos se logran diferenciar más, sobre todo en el derecho (los ranking más altos, de palabras que aparecen menos). También puede deberse a tener datos discretos (menor cantidad) y compararlos contra una serie ajustada que tiene puntos continuos.
De todas formas las desviaciones son esperables, y a grandes rasgos se sigue una ley de potencia entre frecuencia y ranking y por ende, una ley Zipf.

8. *Usando los datos del ejercicio anterior y de acuerdo a la ley de Zipf, calcule la cantidad de palabras que debería haber en el 10%, 20% y 30% del vocabulario. Verifique respecto de los valores reales. Utilice esta aproximación para podar el vocabulario en los mismos porcentajes e indique qué porcentaje de la poda coincide con palabras vacías[13]. Extraiga las palabras podadas que no son stopwords y verifique si, a su criterio, pueden ser importantes para la recuperación.*

Respuesta:
Cantidad de tokens del vocabulario

% vocabulario	Según Zipf	Real observado	Diferencia
10% (2308 términos)	179.895 (50%)	305.470 (85%)	125.574
20% (4617 términos)	194.882 (54%)	324.975 (91%)	130.092
30% (6926 términos)	203648 (57%)	334.561 (93%)	130.912

Tabla 8.1: Tokens del vocabulario según %

Los porcentajes observados distan mucho de los montos que dicta la teoría, sin embargo nos da pista del cumplimiento de la ley de Zipf de igual manera, dado que en sólo el 10% del vocabulario se encuentra el 85% de los tokens, y luego esa medida subirá cada vez más despacio. Se aumenta x2 ese porcentaje (30%) a observar y sin embargo el total de los tokens cubiertos llega al 93%, es decir, sólo 8 puntos más con la medición anterior del 10%.

Poda del vocabulario

% vocabulario	Términos totales	Palabras vacías
10%	2308	153 (6.63%)
20%	4617	170 (3.68%)
30%	6926	181 (2.61%)

Tabla 8.2: Podas al vocabulario según %

[13] Use las palabras vacías provistas en la librería NLTK

Ejemplos de términos:
poda al 10%: 'don', 'si', 'mas', 'quijote', 'sancho', 'dijo', 'tan', 'senor', 'asi', 'respondio', 'ser', 'bien', 'habia', 'merced', 'pues', 'sino', 'dos', 'caballero', 'decir', 'hacer'.

poda al 20%: 'romper', 'nombrar', 'darles', 'advierte', 'venturoso', 'pelea', 'espanol', 'reverencia', 'sandeces', 'cebada', 'compuso', 'coces', 'sentia', 'luengos', 'quisieron', 'anadir', 'tardanza', 'falsa', 'venideros', 'visera'.
poda al 30%: 'escrutinio', 'imaginada', 'recordacion', 'desesperados', 'oida', 'cuenten', 'acaecimientos', 'felicemente', 'gustosos', 'zarandajas', 'desencantar', 'progreso', 'fatigado', 'extraordinario', 'raro', 'intitulado', 'facultad', 'fuésemos', 'ultimamente', 'acordado'.

En la primera poda se obtienen palabras centrales del texto como “quijote”, “sancho”, “caballero”, que tienen gran significado semántico. Si bien no todas son relevantes (“pues”,”dijo”, “bien”) este primer 10% es muy importante para la recuperación.
En la segunda poda seguimos viendo palabras como “pelea”, “español”, “venturoso”, “reverencia” que siguen siendo poco comunes y pueden ser importantes a la hora de asociarse al texto.
Finalmente, ya en la tercera poda se ven pocas palabras verdaderamente relevantes del texto (quizas “escrutinio”, “imaginada”, “zarandajas”), y se notan más generales. Suponiendo posibles consultas de RI, sería más fácil recuperar el texto si las mismas tuvieran “caballero” o “sancho” que si tuvieran “escrutinio”.

9. *Codifique un script que reciba como parámetro el nombre de un archivo de texto, tokenize y calcule y escriba a un archivo los pares (`#términos totales procesados, #términos únicos`). Verifique en qué medida satisface la ley de Heaps. Grafique en la notebook los ajustes variando los parámetros de la expresión. Puede inicialmente probar con los archivos de los puntos anteriores.*

Respuesta
Se realiza una primera iteración del script con el mismo texto de los puntos anteriores (El Quijote de Cervantes). Viéndolo en un gráfico en escala log-log, podemos afirmar que este texto cumple con la Ley de Heaps, ya que a medida que aumenta el número de términos procesados, el número de términos únicos continúa creciendo pero a una tasa menor, formando una relación lineal (los datos reales son los puntos azules).

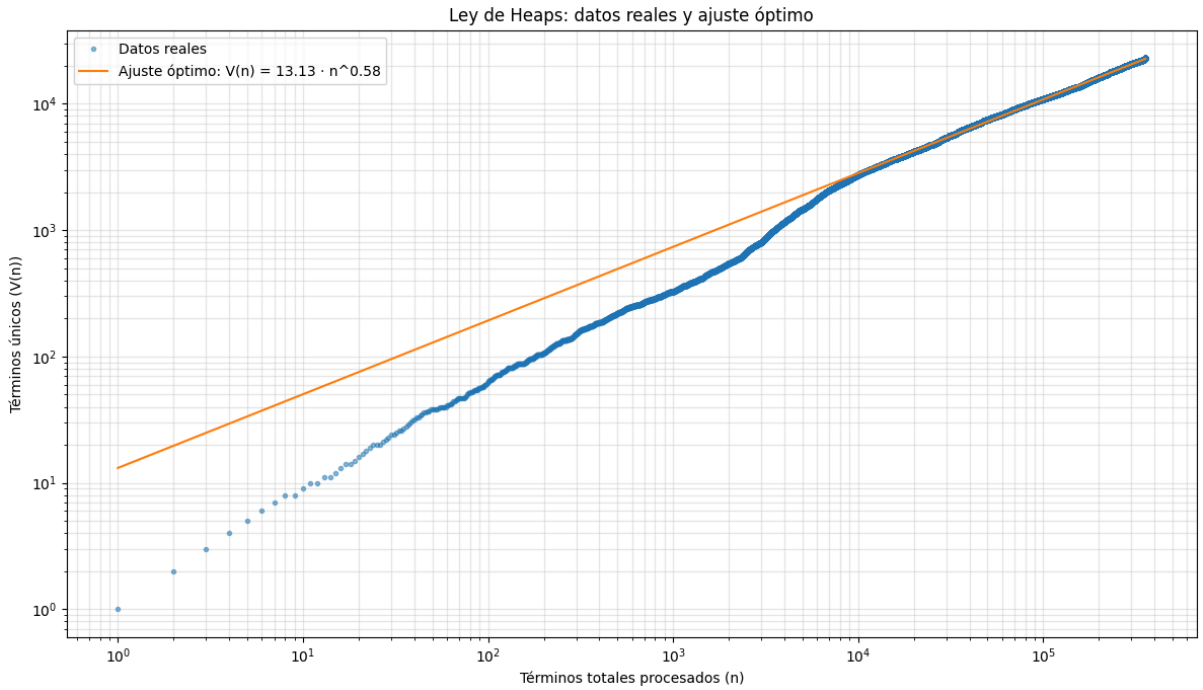

Gráfico 9.1: Datos reales vs ajuste para comprobar Ley de Heaps

Cuando pasamos a los datos ajustados, según Manning[14] y siguiendo esta fórmula:
V(n) = K · nβ
Los valores típicos para los parámetros de la expresión serán los del rango [30, 100] para ***k*** y aprox 0.5 para ***B***, por lo que en el código nos disponemos a probar como se ajusta la curva aumentando ***K*** de 10 en 10, y a ***B*** de a 0.1 entre el rango [0.4, 0.6]. Luego aplicamos la fórmula para las posibles combinaciones.

[14] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Sch¨utze. Introduction to Information Retrieval. Cambridge University Press, New York, NY, USA, 2008. Cap.6.

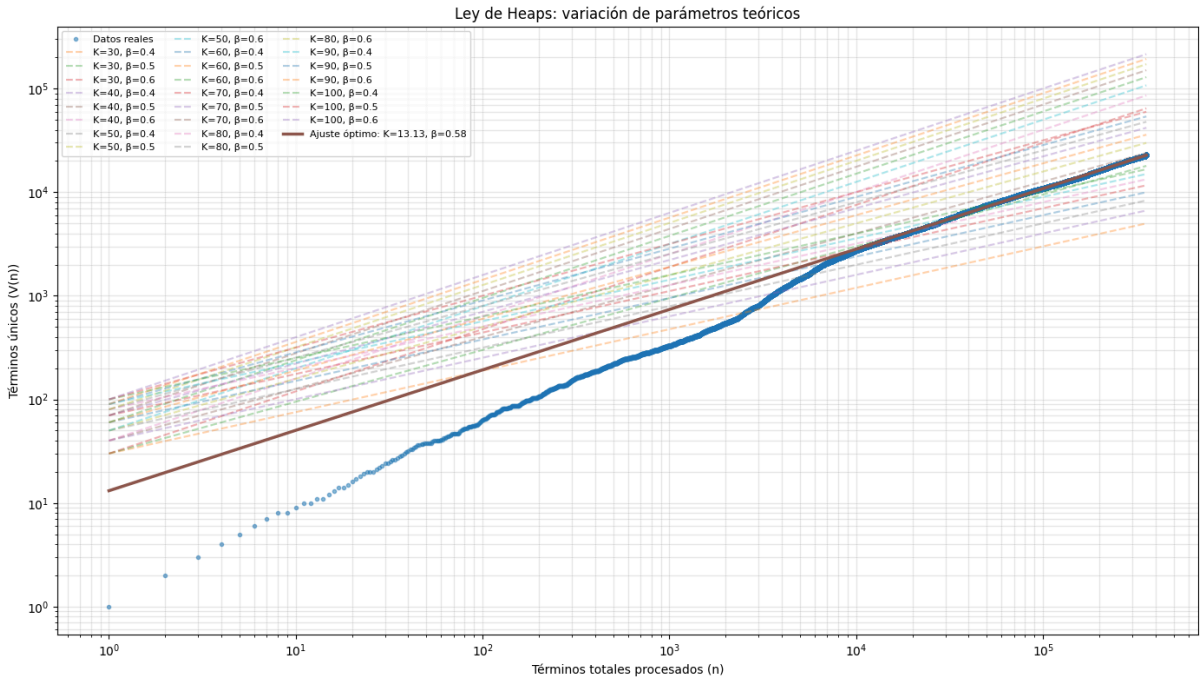

Gráfico 9.2: Variación de parámetros K y B

Vemos que ninguna curva es muy similar a la distribución de datos original (en azul). Luego también tenemos la versión calculada por la librería **curve_fit** del **Gráfico 9.1** (en naranja) que encuentra los parámetros óptimos que mejor se ajustan, la cual nos da unos valores de 13.13 para ***K,*** siendo un número bastante inferior al rango recomendado en la bibliografía, y 0.58 para ***B.***

El valor de ***K*** sólo hace que la curva cambie la posición más arriba o más abajo pero no su forma. Pero ***B*** cumple con las restricciones teóricas (aprox 0.5) lo que sería otro argumento para afirmar que el texto cumple con la Ley de Heaps.